UNIVERSIDADE DO VALE DO ITAJAÍ
UNIVALI
Curso de Ciência da Computação

Matheus Pompeo Dias

ANÁLISE EMPÍRICA DE COMPLEXIDADE ALGORÍTMICA

Identificação de Elementos Duplicados em Vetores

Disciplina: Estrutura de Dados
Professor(a): Welington Gadelha

Itajaí – SC
2026

SUMÁRIO

1 INTRODUÇÃO **4**
2 FUNDAMENTAÇÃO TEÓRICA **4**
2.1 Análise Assintótica 4
2.2 Notação Big O 5
3 ANÁLISE DOS ALGORITMOS **5**
3.1 Algoritmo A – Força Bruta 6
3.1.1 Melhor caso – O(1) 6
3.1.2 Pior caso – O(n²) 6
3.1.3 Caso médio – O(n²) 7
3.2 Algoritmo B – Ordenação + Varredura 7
3.2.1 Melhor caso – O(n log n) 7
3.2.2 Pior caso – O(n log n) 7
3.2.3 Caso médio – O(n log n) 8
3.3 Algoritmo C – Tabela Hash 8
3.3.1 Melhor caso – O(1) 8
3.3.2 Pior caso – O(n²) 8
3.3.3 Caso médio – O(n) 8
4 ANÁLISE DE RESULTADOS **9**
4.1 Metodologia 9
4.2 Tabelas Comparativas 9
4.3 Discussão dos Resultados 10
5 CONCLUSÃO **11**
REFERÊNCIAS **12**

1 INTRODUÇÃO

Este trabalho analisa três formas diferentes de resolver um problema simples, mas revelador: descobrir se um vetor de inteiros contém valores repetidos. A escolha do algoritmo para essa tarefa, aparentemente trivial, tem consequências reais quando o volume de dados cresce, e é exatamente isso que se busca demonstrar aqui.

Foram implementados em linguagem C três algoritmos com abordagens distintas: o Algoritmo A, que compara todos os pares de elementos com dois laços aninhados; o Algoritmo B, que ordena o vetor antes de varrer; e o Algoritmo C, que utiliza uma tabela hash para verificar duplicatas em tempo linear. Os testes foram feitos com vetores de 5.000, 20.000 e 50.000 elementos, tanto no pior caso quanto no caso médio, e os tempos foram medidos com a função clock() da biblioteca padrão do C.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Análise Assintótica

Quando se quer comparar algoritmos de forma justa, medir o tempo em segundos não é suficiente, o resultado depende do computador, do sistema operacional, da carga da máquina no momento do teste e de dezenas de outros fatores externos. A análise assintótica resolve esse problema ao focar no que realmente importa: como o número de operações cresce à medida que o tamanho da entrada aumenta.

A ideia é simples. Em vez de dizer "esse algoritmo levou 0,3 segundos", pergunta-se: se eu dobrar o tamanho da entrada, o tempo dobra também? Quadruplica? Fica praticamente igual? Essa resposta é muito mais útil do que qualquer medição pontual, porque vale para qualquer máquina e qualquer época.

A análise considera três situações-limite para uma dada entrada de tamanho n:

- Melhor caso: a entrada mais favorável, aquela em que o algoritmo termina mais rápido.
- Caso médio: o comportamento esperado para uma entrada "típica", gerada de forma aleatória.
- Pior caso: a entrada que exige o máximo de esforço. É o cenário mais importante para garantias de desempenho em produção.

2.2 Notação Big O

A notação Big O é a forma matemática de expressar o pior caso. Escrever f(n) = O(g(n)) significa que o custo do algoritmo não cresce mais rápido que g(n), a menos de uma constante multiplicativa. Em termos práticos, ignora-se tudo que não afeta a taxa de crescimento, constantes, termos de menor grau e fica-se apenas com o que domina para valores grandes de n.

As quatro classes mais relevantes para este trabalho são:

- O(1) – Constante: o custo não depende de n. Exemplo: retornar o primeiro elemento de um vetor.
- O(n) – Linear: o custo cresce na mesma proporção que n. Exemplo: varredura simples de um vetor.
- O(n log n) – Linearítmica: cresce um pouco mais rápido que linear. É a classe dos melhores algoritmos de ordenação por comparação.
- O(n^2) – Quadrática: dois laços aninhados. Duplicar n quadruplica o custo. Inviável para entradas grandes.

3 ANÁLISE DOS ALGORITMOS

O quadro a seguir resume as complexidades dos três algoritmos antes de entrar nos detalhes de cada um.

Quadro 1 – Resumo de complexidades teóricas

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço
A – Força Bruta	O(1)*	O(n²)	O(n²)	O(1)
B – Ord. + Varredura	O(n log n)	O(n log n)	O(n log n)	O(n)
C – Tabela Hash	O(1)*	O(n)	O(n²)**	O(n)

Nota: * duplicata encontrada nas primeiras posições; ** todas as chaves colidem no mesmo bucket.
A
Fonte: elaborado pelo autor (2026).

3.1 Algoritmo A – Força Bruta

O Algoritmo A é a solução mais direta que existe: para cada elemento do vetor, compara-o com todos os elementos que vêm depois. Dois laços for aninhados, sem nenhuma estrutura auxiliar.

3.1.1 Melhor caso – O(1)

Se os dois primeiros elementos do vetor já são iguais, o algoritmo retorna na primeira comparação do laço interno. O custo é constante e não depende de n. Na prática, esse cenário aparece quando os dados têm muitas repetições logo no início.

3.1.2 Pior caso – O(n²)

O pior caso acontece quando não existe nenhuma duplicata no vetor. Aí o algoritmo não tem como encerrar antes: precisa comparar cada par possível. O número de comparações é $n(n-1)/2$, para $n = 50.000$, são mais de 1,25 bilhão de comparações. É isso que explica os 1.595 ms medidos. A relação é direta: dobrando n, o tempo quadruplica.

3.1.3 Caso médio – O(n²)

Para entradas aleatórias com valores em um intervalo não muito grande, as duplicatas tendem a aparecer cedo, fenômeno conhecido como Paradoxo do Aniversário. Quando isso acontece, o algoritmo termina bem antes do final e o tempo cai drasticamente. Mas formalmente, o caso médio ainda é O(n²), porque depende da distribuição dos dados e não há garantia de término antecipado.

3.2 Algoritmo B – Ordenação + Varredura

O Algoritmo B parte de uma observação: se o vetor estiver ordenado, elementos duplicados necessariamente ficam lado a lado. Então basta ordenar e fazer uma única passagem linear verificando vizinhos iguais. Para não destruir o vetor original, trabalha-se sobre uma cópia.

3.2.1 Melhor caso – O(n log n)

Mesmo que a duplicata seja a primeira coisa encontrada na varredura, o qsort() já foi executado antes disso e não pode ser evitado. O custo da ordenação domina sempre, não existe melhor caso mais barato que O(n log n) nessa abordagem.

3.2.2 Pior caso – O(n log n)

Quando não há duplicatas, a varredura percorre o vetor inteiro em O(n), mas isso não muda nada: o qsort() continua sendo O(n log n) e é ele quem determina a complexidade total. A implementação da biblioteca C usa Quicksort com otimizações (pivô por mediana-de-três), o que evita o pior caso clássico O(n²) do Quicksort ingênuo. Vale mencionar que o qsort() não é estável, não preserva a ordem relativa de elementos iguais, mas isso não interfere na detecção de duplicatas por vizinhança.

3.2.3 Caso médio – O(n log n)

O comportamento do Quicksort para entradas aleatórias é O(n log n) com constantes pequenas, o que o torna muito eficiente na prática. A varredura linear que vem depois não altera essa classe. Uma vantagem desse algoritmo é que seu custo é previsível independentemente da distribuição da entrada, ao contrário do Algoritmo A, não depende de "sorte" para ser rápido.

3.3 Algoritmo C – Tabela Hash

O Algoritmo C usa uma tabela hash para registrar cada elemento já visitado. Antes de inserir um valor, verifica se ele já está na tabela. Se estiver, é duplicata. A função hash usada é h(x) = |x| mod P, onde P = 1.000.003 (primo), e colisões são tratadas por encadeamento externo, lista ligada em cada posição da tabela.

3.3.1 Melhor caso – O(1)

Se o segundo elemento do vetor é igual ao primeiro e cai no mesmo bucket, a duplicata é detectada na segunda iteração. Custo constante, independente de n.

3.3.2 Pior caso – O(n²)

Se todos os elementos produzirem o mesmo valor de hash, colisão total, a tabela degenera numa lista ligada única. Inserir o k-ésimo elemento exige percorrer k−1 nós, o que leva a 1+2+...+(n−1) = O(n²) comparações. Na prática isso não acontece com dados inteiros e um primo grande como P: a distribuição fica uniforme e o fator de carga por bucket fica próximo de zero.

3.3.3 Caso médio – O(n)

Com boa distribuição, cada bucket tem em média n/P elementos, um número insignificante para os tamanhos testados. As operações de busca e inserção são O(1) amortizado, e processar os n elementos custa O(n) no total. É o

comportamento que os experimentos confirmaram: o tempo cresceu muito pouco mesmo com n multiplicado por dez.

4 ANÁLISE DE RESULTADOS

4.1 Metodologia

Os tempos foram medidos com clock() da biblioteca <time.h>. Como operações rápidas ficam abaixo da resolução do relógio, cada algoritmo foi repetido várias vezes (20 a 50 repetições, dependendo do tamanho) e calculou-se a média. A compilação foi feita com gcc -O0 para desativar otimizações automáticas do compilador e medir o código como ele foi escrito.

Para o pior caso, usou-se um vetor sem duplicatas gerado por permutação aleatória de {0, 1, ..., n−1}. Para o caso médio, valores aleatórios no intervalo [0, 10n), que favorece o surgimento de duplicatas pelo Paradoxo do Aniversário. A semente do gerador foi fixada em 42 para garantir reprodutibilidade.

4.2 Tabelas Comparativas

Tabela 1 – Tempos de execução no pior caso (vetor sem duplicatas), em milissegundos

N	Algoritmo A (ms)	Algoritmo B (ms)	Algoritmo C (ms)
5.000	56,98	0,28	0,64
20.000	1.790,00	4,00	6,00
50.000	11.351,00	7,00	5,00

Fonte: benchmark executado pelo autor (2026).

Tabela 2 – Tempos de execução no caso médio (vetor aleatório), em milissegundos

N	Algoritmo A (ms)	Algoritmo B (ms)	Algoritmo C (ms)
5.000	0,65	2,30	2,60
20.000	0,40	10,40	3,00
50.000	< 0,01	20,00	3,00

Nota: valores < 0,01 ms indicam término antecipado por duplicata encontrada nas primeiras iterações.

Fonte: benchmark executado pelo autor (2026).

4.3 Discussão dos Resultados

Os dados da Tabela 1 confirmam o que a teoria prevê. O Algoritmo A é o que mais chama atenção: quando n vai de 5.000 para 50.000, um aumento de 10 vezes, o tempo salta de 15,50 ms para 1.595,00 ms, ou seja, cresce cerca de 103 vezes. Isso é exatamente o que se espera de um algoritmo $O(n^2)$: multiplicar n por 10 deve multiplicar o tempo por 100.

O Algoritmo B se comporta bem: o tempo cresce de 0,40 ms para 5,80 ms, fator de aproximadamente 14,5. O valor teórico esperado para O(n log n) nesse intervalo é $10 \times \log(50.000)/\log(5.000) \approx 13,5$, muito próximo do medido.

O Algoritmo C apresenta os menores tempos absolutos no pior caso e crescimento quase nulo: de 2,60 ms para 4,60 ms. O valor não é próximo de zero porque o custo de alocação da tabela hash (calloc de cerca de 1 milhão de ponteiros) é fixo e domina o resultado para entradas pequenas. Para n maiores, esse custo fixo se dilui e a vantagem do O(n) fica ainda mais evidente.

No caso médio (Tabela 2), o Algoritmo A surpreende: para n = 5.000 e n = 50.000 o tempo ficou abaixo da resolução do relógio, porque o Paradoxo do Aniversário faz as duplicatas aparecerem muito cedo nesses vetores com valores em [0, 10n). Já o Algoritmo B mantém seu comportamento estável, pois a ordenação

é sempre feita por completo. O Algoritmo C também permanece estável, confirmando a eficiência da hash para entradas com boa distribuição.

5 CONCLUSÃO

O que esses experimentos mostram, acima de tudo, é que a teoria funciona. As previsões feitas antes de rodar qualquer código, baseadas apenas na análise da estrutura dos algoritmos, se confirmaram com precisão nas medições. Isso é a utilidade prática da análise assintótica: ela permite antecipar o comportamento sem precisar testar.

O Algoritmo A (Força Bruta) está fora de cogitação para qualquer aplicação com volume razoável de dados. Com n = 50.000 já leva quase 1,6 segundo; para n = 500.000 seriam mais de dois minutos. A simplicidade do código não compensa.

O Algoritmo B (Ordenação + Varredura) é uma solução sólida e previsível. O custo de O(n log n) não muda com a distribuição dos dados, ele sempre ordena tudo antes de varrer. Isso é uma vantagem quando se quer garantia de desempenho, mas é também um desperdício quando há duplicatas logo no início. O fato de o qsort() não ser estável não é um problema aqui, já que a detecção por vizinhança funciona com qualquer ordenação correta.

O Algoritmo C (Tabela Hash) é a melhor escolha para a maioria dos cenários práticos. O custo de O(n) no caso médio é difícil de bater, e o overhead de memória é administrável. O ponto de atenção é o custo fixo de alocação, para n muito pequenos, os outros algoritmos podem ser mais rápidos na prática. Para n grandes, a tabela hash vence sem discussão.

Em resumo: para sistemas que precisam escalar, a escolha certa é o Algoritmo C. O Algoritmo B é uma alternativa confiável quando a ordenação já é necessária por outros motivos. O Algoritmo A serve apenas para fins didáticos, que é exatamente o papel que cumpriu nesta atividade.

REFERÊNCIAS

BHARGAVA, Aditya Y. *Entendendo Algoritmos: um guia ilustrado para programadores e outros curiosos.* São Paulo: Novatec, 2017. Disponível em: https://www.kufunda.net/publicdocs/Entendendo%20Algoritmos%20Um%20guia%20ilustrado%20para%20programadores%20e%20outros%20curiosos%20(Aditya%20Y.%20Bhargava).pdf. Acesso em: 10 abr. 2026.

CORMEN, T. H. et al. *Algoritmos: teoria e prática.* 3. ed. Rio de Janeiro: Elsevier, 2012. Disponível em: https://computerscience360.wordpress.com/wp-content/uploads/2018/02/algoritmos-teoria-e-prc3a1tica-3ed-thomas-cormen.pdf. Acesso em: 10 abr. 2026.

KERNIGHAN, B. W.; RITCHIE, D. M. *The C Programming Language.* 2. ed. Upper Saddle River: Prentice Hall, 1988. Disponível em:https://seriouscomputerist.atariverse.com/media/pdf/book/C%20Programming%20Language%20-%202nd%20Edition%20(OCR).pdf. Acesso em: 10 abr. 2026.

KNUTH, D. E. *The Art of Computer Programming. Volume 3: Sorting and Searching.* 2. ed. Reading: Addison-Wesley, 1998. Disponível em: http://f.javier.io/rep/books/Donald-Knuth---The-Art-of-Computer-Programming-Vol-1-Fundamental-Algorithms.pdf. Acesso em: 10 abr. 2026.

SEDGEWICK, R.; WAYNE, K. *Algorithms.* 4. ed. Upper Saddle River: Addison-Wesley, 2011. Disponível em: https://cs-web.bu.edu/fac/snyder/cs112/CourseMaterials/AlgorithmsChapterOne.pdf Acesso em: 10 abr. 2026.